OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

Span CLASS

SOURCE

A slice from a `Doc` object.

Span.__init__ METHOD

Create a `Span` object from the slice `doc[start : end]`.

NAME	DESCRIPTION
`doc`	The parent document. **TYPE:** `Doc`
`start`	The index of the first token of the span. **TYPE:** `int`
`end`	The index of the first token after the span. **TYPE:** `int`
`label`	A label to attach to the span, e.g. for named entities. **TYPE:** `Union[str, int]`
`vector`	A meaning representation of the span. **TYPE:** `numpy.ndarray[ndim=1, dtype=float32]`
`vector_norm`	The L2 norm of the document's vector representation. **TYPE:** `float`
`kb_id`	A knowledge base ID to attach to the span, e.g. for named entities. **TYPE:** `Union[str, int]`
`span_id`	An ID to associate with the span. **TYPE:** `Union[str, int]`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[1:4]
assert [t.text for t in span] == ["it", "back", "!"]
```

Span.__getitem__ METHOD

Get a `Token` object.

NAME	DESCRIPTION
`i`	The index of the token within the span. **TYPE:** `int`
RETURNS	The token at `span[i]`.

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[1:4]
assert span[1].text == "back"
```

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

TYPE: Token

Get a `Span` object.

NAME	DESCRIPTION
`start_end`	The slice of the span to get. **TYPE:** `Tuple[int, int]`
RETURNS	The span at `span[start : end]`. **TYPE:** `Span`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[1:4]
assert span[1:3].text == "back!"
```

Span.__iter__ METHOD

Iterate over `Token` objects.

NAME	DESCRIPTION
YIELDS	A `Token` object. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[1:4]
assert [t.text for t in span] == ["it", "back", "!"]
```

Span.__len__ METHOD

Get the number of tokens in the span.

NAME	DESCRIPTION
RETURNS	The number of tokens in the span. **TYPE:** `int`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[1:4]
assert len(span) == 3
```

Span.set_extension CLASSMETHOD

Define a custom attribute on the `Span` which becomes available via `Span.._`. For details, see the documentation on custom attributes.

NAME	DESCRIPTION
`name`	Name of the attribute to set by the extension. For example, `"my_attr"` will be available as `span._.my_attr`. **TYPE:** `str`
`default`	Optional default value of the attribute if no getter or method is defined.

EXAMPLE

```
from spacy.tokens import Span
city_getter = lambda span: any(city in span.text for city in ("New York"
Span.set_extension("has_city", getter=city_getter)
```

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

	TYPE: `Optional[Any]`
`method`	Set a custom method on the object, for example `span._.compare(other_span)`.
	TYPE: `Optional[Callable[[Span, …], Any]]`
`getter`	Getter function that takes the object and returns an attribute value. Is called when the user accesses the `._` attribute.
	TYPE: `Optional[Callable[[Span], Any]]`
`setter`	Setter function that takes the `Span` and a value, and modifies the object. Is called when the user writes to the `Span._` attribute.
	TYPE: `Optional[Callable[[Span, Any], None]]`
`force`	Force overwriting existing attribute.
	TYPE: `bool`

Span.get_extension CLASSMETHOD

Look up a previously registered extension by name. Returns a 4-tuple `(default, method, getter, setter)` if the extension is registered. Raises a `KeyError` otherwise.

NAME	DESCRIPTION
`name`	Name of the extension.
	TYPE: `str`
RETURNS	A `(default, method, getter, setter)` tuple of the extension.
	TYPE: `Tuple[Optional[Any], Optional[Callable], Optional[Callable], Optional[Callable]]`

EXAMPLE

```
from spacy.tokens import Span
Span.set_extension("is_city", default=False)
extension = Span.get_extension("is_city")
assert extension == (False, None, None, None)
```

Span.has_extension CLASSMETHOD

Check whether an extension has been registered on the `Span` class.

NAME	DESCRIPTION
`name`	Name of the extension to check.
	TYPE: `str`
RETURNS	Whether the extension has been registered.
	TYPE: `bool`

EXAMPLE

```
from spacy.tokens import Span
Span.set_extension("is_city", default=False)
assert Span.has_extension("is_city")
```

Span.remove_extension CLASSMETHOD

Remove a previously registered extension.

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

name	Name of the extension. **TYPE:** str
RETURNS	A (default, method, getter, setter) tuple of the removed extension. **TYPE:** Tuple[Optional[Any], Optional[Callable], Optional[Callable], Optional[Callable]]

```
Span.set_extension("is_city", default=False)
removed = Span.remove_extension("is_city")
assert not Span.has_extension("is_city")
```

Span.char_span METHOD

Create a Span object from the slice span.text[start:end]. Returns None if the character indices don't map to a valid span.

NAME	DESCRIPTION
start	The index of the first character of the span. **TYPE:** int
end	The index of the last character after the span. **TYPE:** int
label	A label to attach to the span, e.g. for named entities. **TYPE:** Union[int, str]
kb_id	An ID from a knowledge base to capture the meaning of a named entity. **TYPE:** Union[int, str]
vector	A meaning representation of the span. **TYPE:** numpy.ndarray[ndim=1, dtype=float32]
id	Unused. **TYPE:** Union[int, str]
alignment_mode V3.5.1	How character indices snap to token boundaries. Options: "strict" (no snapping), "contract" (span of all tokens completely within the character span), "expand" (span of all tokens at least partially covered by the character span). Defaults to "strict". **TYPE:** str
span_id V3.5.1	An identifier to associate with the span. **TYPE:** Union[int, str]
RETURNS	The newly constructed object or None. **TYPE:** Optional[Span]

EXAMPLE

```
doc = nlp("I like New York")
span = doc[1:4].char_span(5, 13, label="GPE")
assert span.text == "New York"
```

Span.similarity METHOD NEEDS MODEL

Make a semantic similarity estimate. The default estimate is cosine similarity using an average of word vectors.

NAME	DESCRIPTION
other	The object to compare with. By default, accepts Doc, Span, Token and Lexeme objects. **TYPE:** Union[Doc,Span,Token,Lexeme]

EXAMPLE

```
doc = nlp("green apples and red oranges")
green_apples = doc[:2]
red_oranges = doc[3:]
apples_oranges = green_apples.similarity(red_oranges)
```

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

RETURNS	A scalar similarity score. Higher is more similar. **TYPE:** `float`

Span.get_lca_matrix METHOD

Calculates the lowest common ancestor matrix for a given `Span`. Returns LCA matrix containing the integer index of the ancestor, or `-1` if no common ancestor is found, e.g. if span excludes a necessary ancestor.

NAME	DESCRIPTION
RETURNS	The lowest common ancestor matrix of the `Span`. **TYPE:** `numpy.ndarray[ndim=2, dtype=int32]`

EXAMPLE

```
doc = nlp("I like New York in Autumn")
span = doc[1:4]
matrix = span.get_lca_matrix()
# array([[0, 0, 0], [0, 1, 2], [0, 2, 2]], dtype=int32)
```

Span.to_array METHOD

Given a list of `M` attribute IDs, export the tokens to a numpy `ndarray` of shape `(N, M)`, where `N` is the length of the document. The values will be 32-bit integers.

NAME	DESCRIPTION
`attr_ids`	A list of attributes (int IDs or string names) or a single attribute (int ID or string name). **TYPE:** `Union[int, str, List[Union[int, str]]]`
RETURNS	The exported attributes as a numpy array. **TYPE:** `Union[numpy.ndarray[ndim=2, dtype=uint64],numpy.ndarray[ndim=1, dtype=uint64]]`

EXAMPLE

```
from spacy.attrs import LOWER, POS, ENT_TYPE, IS_ALPHA
doc = nlp("I like New York in Autumn.")
span = doc[2:3]
# All strings mapped to integers, for easy export to numpy
np_array = span.to_array([LOWER, POS, ENT_TYPE, IS_ALPHA])
```

Span.ents PROPERTY NEEDS MODEL

The named entities that fall completely within the span. Returns a tuple of `Span` objects.

NAME	DESCRIPTION
RETURNS	Entities in the span, one `Span` per entity. **TYPE:** `Tuple[Span, ...]`

EXAMPLE

```
doc = nlp("Mr. Best flew to New York on Saturday morning.")
span = doc[0:6]
ents = list(span.ents)
assert ents[0].label == 346
assert ents[0].label_ == "PERSON"
assert ents[0].text == "Mr. Best"
```

Span.noun_chunks PROPERTY NEEDS MODEL

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

If the `noun_chunk` syntax iterator has not been implemeted for the given language, a `NotImplementedError` is raised.

NAME	DESCRIPTION
YIELDS	Noun chunks in the span. **TYPE:** Span

EXAMPLE

```
doc = nlp("A phrase with another phrase occurs.")
span = doc[3:5]
chunks = list(span.noun_chunks)
assert len(chunks) == 1
assert chunks[0].text == "another phrase"
```

Span.as_doc METHOD

Create a new `Doc` object corresponding to the `Span`, with a copy of the data.

When calling this on many spans from the same doc, passing in a precomputed array representation of the doc using the `array_head` and `array` args can save time.

NAME	DESCRIPTION
`copy_user_data`	Whether or not to copy the original doc's user data. **TYPE:** `bool`
`array_head`	Precomputed array attributes (headers) of the original doc, as generated by `Doc._get_array_attrs()`. **TYPE:** `Tuple`
`array`	Precomputed array version of the original doc as generated by `Doc.to_array`. **TYPE:** `numpy.ndarray`
RETURNS	A `Doc` object of the `Span`'s content. **TYPE:** `Doc`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
span = doc[2:4]
doc2 = span.as_doc()
assert doc2.text == "New York"
```

Span.root PROPERTY NEEDS MODEL

The token with the shortest path to the root of the sentence (or the root itself). If multiple tokens are equally high in the tree, the first token is taken.

NAME	DESCRIPTION
RETURNS	The root token. **TYPE:** Token

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
i, like, new, york, in_, autumn, dot = range(len(doc))
assert doc[new].head.text == "York"
assert doc[york].head.text == "like"
new_york = doc[new:york+1]
assert new_york.root.text == "York"
```

Span.conjuncts PROPERTY NEEDS MODEL

A tuple of tokens coordinated to `span.root`.

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

RETURNS	The coordinated tokens. **TYPE:** `Tuple[Token, …]`

```
apples_conjuncts = doc[2:3].conjuncts
assert [t.text for t in apples_conjuncts] == ["oranges"]
```

Span.lefts PROPERTY NEEDS MODEL

Tokens that are to the left of the span, whose heads are within the span.

NAME	DESCRIPTION
YIELDS	A left-child of a token of the span. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
lefts = [t.text for t in doc[3:7].lefts]
assert lefts == ["New"]
```

Span.rights PROPERTY NEEDS MODEL

Tokens that are to the right of the span, whose heads are within the span.

NAME	DESCRIPTION
YIELDS	A right-child of a token of the span. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
rights = [t.text for t in doc[2:4].rights]
assert rights == ["in"]
```

Span.n_lefts PROPERTY NEEDS MODEL

The number of tokens that are to the left of the span, whose heads are within the span.

NAME	DESCRIPTION
RETURNS	The number of left-child tokens. **TYPE:** `int`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
assert doc[3:7].n_lefts == 1
```

Span.n_rights PROPERTY NEEDS MODEL

The number of tokens that are to the right of the span, whose heads are within the span.

EXAMPLE

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

RETURNS	The number of right-child tokens. **TYPE:** `int`

Span.subtree PROPERTY NEEDS MODEL

Tokens within the span and tokens which descend from them.

NAME	DESCRIPTION
YIELDS	A token within the span, or a descendant from it. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
subtree = [t.text for t in doc[:3].subtree]
assert subtree == ["Give", "it", "back", "!"]
```

Span.has_vector PROPERTY NEEDS MODEL

A boolean value indicating whether a word vector is associated with the object.

NAME	DESCRIPTION
RETURNS	Whether the span has a vector data attached. **TYPE:** `bool`

EXAMPLE

```
doc = nlp("I like apples")
assert doc[1:].has_vector
```

Span.vector PROPERTY NEEDS MODEL

A real-valued meaning representation. Defaults to an average of the token vectors.

NAME	DESCRIPTION
RETURNS	A 1-dimensional array representing the span's vector. **TYPE:** `numpy.ndarray[ndim=1, dtype=float32]`

EXAMPLE

```
doc = nlp("I like apples")
assert doc[1:].vector.dtype == "float32"
assert doc[1:].vector.shape == (300,)
```

Span.vector_norm PROPERTY NEEDS MODEL

The L2 norm of the span's vector representation.

EXAMPLE

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

RETURNS	The L2 norm of the vector representation. **TYPE:** `float`

```
assert doc[1:].vector_norm != doc[2:].vector_norm
```

Span.sent PROPERTY NEEDS MODEL

The sentence span that this span is a part of. This property is only available when sentence boundaries have been set on the document by the `parser`, `senter`, `sentencizer` or some custom function. It will raise an error otherwise.

If the span happens to cross sentence boundaries, only the first sentence will be returned. If it is required that the sentence always includes the full span, the result can be adjusted as such:

```
sent = span.sent
sent = doc[sent.start : max(sent.end, span.end)]
```

NAME	DESCRIPTION
RETURNS	The sentence span that this span is a part of. **TYPE:** `Span`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[1:3]
assert span.sent.text == "Give it back!"
```

Span.sents PROPERTY V3.2.1 NEEDS MODEL

Returns a generator over the sentences the span belongs to. This property is only available when sentence boundaries have been set on the document by the `parser`, `senter`, `sentencizer` or some custom function. It will raise an error otherwise.

If the span happens to cross sentence boundaries, all sentences the span overlaps with will be returned.

NAME	DESCRIPTION
RETURNS	A generator yielding sentences this `Span` is a part of **TYPE:** `Iterable[Span]`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
span = doc[2:4]
assert len(span.sents) == 2
```

Attributes

NAME	DESCRIPTION
`doc`	The parent document. **TYPE:** `Doc`
`tensor`	The span's slice of the parent `Doc`'s tensor.

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

`start`	The token offset for the start of the span. **TYPE:** `int`
`end`	The token offset for the end of the span. **TYPE:** `int`
`start_char`	The character offset for the start of the span. **TYPE:** `int`
`end_char`	The character offset for the end of the span. **TYPE:** `int`
`text`	A string representation of the span text. **TYPE:** `str`
`text_with_ws`	The text content of the span with a trailing whitespace character if the last token has one. **TYPE:** `str`
`orth`	ID of the verbatim text content. **TYPE:** `int`
`orth_`	Verbatim text content (identical to `Span.text`). Exists mostly for consistency with the other attributes. **TYPE:** `str`
`label`	The hash value of the span's label. **TYPE:** `int`
`label_`	The span's label. **TYPE:** `str`
`lemma_`	The span's lemma. Equivalent to `"".join(token.text_with_ws for token in span)`. **TYPE:** `str`
`kb_id`	The hash value of the knowledge base ID referred to by the span. **TYPE:** `int`
`kb_id_`	The knowledge base ID referred to by the span. **TYPE:** `str`
`ent_id`	The hash value of the named entity the root token is an instance of. **TYPE:** `int`
`ent_id_`	The string ID of the named entity the root token is an instance of. **TYPE:** `str`
`id`	The hash value of the span's ID. **TYPE:** `int`
`id_`	The span's ID. **TYPE:** `str`
`sentiment`	A scalar value indicating the positivity or negativity of the span. **TYPE:** `float`
`_`	User space for adding custom attribute extensions. **TYPE:** `Underscore`

OVERVIEW
Library Architecture
Model Architectures
Data Formats
Command Line
Functions

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer

SUGGEST EDITS

SPACY
Usage
Models
API Reference
Online Course
Custom Solutions

COMMUNITY
Universe
GitHub Discussions
Issue Tracker
Stack Overflow

CONNECT
Twitter
GitHub
YouTube
Blog

STAY IN THE LOOP!
Receive updates about new releases, tutorials and more.

Your email SIGN UP

© 2016-2023 Explosion Legal / Imprint